

Exercise 2

Analysis:

Applications is built of 2 functions:

makeEncodedFile: takes file name, name for new file, and value of K

- import wave and use that library to open wav file and read all the bytes from the file and then change them to list of ints.
- Inner function encode takes a single int, and k value and returns encoded string,
- Inner function encodeList, takes list and K, that function iterates over each item of the list, which is an int, calls encode on every item and returns encoded binary string
- Inner function binaryStrToBytes turns binary string to bytes to preserve space
- Using wave library create new file, write to it set parameter and encoded byte object

makeDecodedFile:

- Use wave library to get file data first as bytes turn into list of ints
- Turn int list into one binary string
- Inner function decode takes single encoded value, and K, returns decoded number
- Inner function decodeStr, takes entire binary string and value K, (takes first 0s and 255 out more in farther development), then counts unary part and K bits to set the index for each separate chunk to be decoded and feeds that info to decode function resulting in decoded int list
- Change int list to bytes and create new file, write to it set parameters and decoded byte object

Table:

	Original size	Rice (K = 4 bits)	Rice (K = 2 bits)	% Compression (K = 4 bits)	% Compression (K = 2 bits)
Sound1.wav	1 MB	1.5 MB	4.1 MB	150%	410%
Sound2.wav	1 MB	1.5 MB	4.3 MB	150%	430%

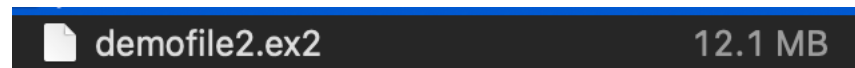
Results were different because quotient is encoded in unary the higher the quotient the more bits will be taken to store it. Where reminder has the size of K bits, with K = 2 to 2bts were saved per each value, but to keep encoding lossless that highly effected the unary size therefore the numbers got so much bigger.

Sound1 K=2 is taking less space because seems like there were more lower values ints so that made sizes of unary quotient smaller.

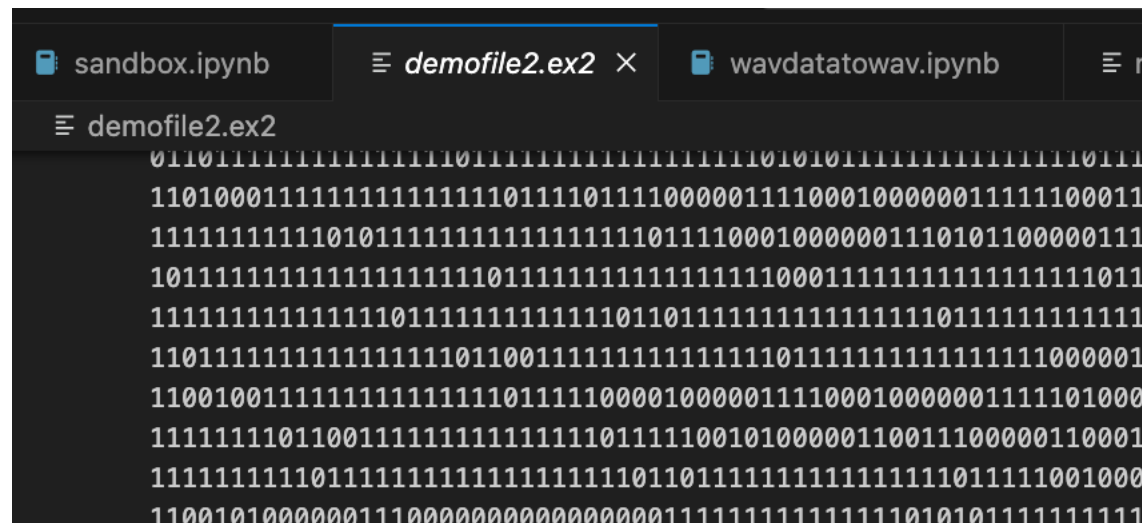
Brief description of farther development:

1. Adding structure to encoded file

First approach I encoded Int list into binary string and tried to store that binary string, that did not work my file ended up being 12mb from 1mb



Looked like this:



So, I decided that if I add some structure to turn it into byte object I could store it better. Problem was that unary was unpredictable with number of bits and I needed amount of bits to be dividable by 8. I decided to add max value of 255 at the beginning and fill out the '0' before that to make string dividable by 8. This allowed me to make sure that decoding is going to be lossless to original file.

2. Decoding efficiency

When decoding I tried turning list of ints into binary string I tried looping over every item in the list after over a minute of code running I decided I will try different approaches to improve efficiency.

```
encodedBinaryStr = ''
for item in encodedSoundIntList:
    encodedBinaryStr = encodedBinaryStr + format(item, '08b')

121] [X] 1m 10.6s

-----
KeyboardInterrupt                                Traceback (most recent call last)
Cell In[121], line 11
      9 encodedBinaryStr = ''
     10 for item in encodedSoundIntList:
--> 11     encodedBinaryStr = encodedBinaryStr + format(item, '08b')

KeyboardInterrupt:
```

I tried changing list of ints to list of binary values and then joined it together into string. That change speed of that code to 0.2 s.

```
#first take a file
encodedSound = wave.open("Exercise2_Files/test1.wav", mode='rb')
# Extract all the bytes
encodedSoundBytes = encodedSound.readframes(-1) # data is in byte object type
encodedSoundParameters = encodedSound.getparams()
encodedSound.close()

encodedSoundIntList = list(encodedSoundBytes) # data is the list of ints
encodedBinaryList = []
for i in range(len(encodedSoundIntList)):
    encodedBinaryList.append(format(encodedSoundIntList[i], '08b'))
encodedBinaryStr = ''.join(encodedBinaryList)

[133] [✓] 0.2s
```